Friedrich-Alexander-Universität
Technische Fakultät

Pattern Recognition Lab

FAU

Introduction to Software Engineering

Architectural Design

A. Maier, A. Sindel, S. Zeitler

Pattern Recognition Lab, Friedrich-Alexander-Universität Erlangen-Nürnberg

November 11, 2024

Software architecture[1]

Software architecture can be defined as:

> The fundamental organization of a system embodied in its components, their relationships to each other, and to the environment, and the principles guiding its design and evolution [1]

As such is contains:

- the **static** disassembly of a system into it's components
- the description of the **dynamic** interaction of all components
- the strategy of the architecture

[1] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 5

Software architecture[2]

The architecture of a software is highly interleaved with requirements engineering as the architecture should:

- meet functional requirements
- meet non-functional requirements (quality, performance, etc.)
- provide a blueprint for the system implementation

Both risk and development cost of a project can be reduced by the use of patterns.

A complete system architecture can combine multiple architectural patterns.

[2] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 3-6

1. Architectural design decisions

2. Architectural patterns

2.1 Structuring patterns
2.2 Adaptable systems
2.3 Distributed systems
2.4 Interactive systems

1. Architectural design decisions

2. Architectural patterns

2.1 Structuring patterns
2.2 Adaptable systems
2.3 Distributed systems
2.4 Interactive systems

Architectural design decisions

Architectural design decisions[3,4]

The non-functional requirements that should be considered for the system architecture are

- Performance
- Security
- Safety
- Availability
- Maintainability
- Simplicity
- Scalability
- Testability
- Expandability
- Reliability (tested and proven architectures)

[3] Sommerville: *Software Engineering* (2016) [3] p. 172-173

[4] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 6-7

Architectural design decisions[5]

To meet these requirements, the architect has to decide on:

Technical aspects

- System structure
- Interfaces
- Dynamic behavior
- Patterns
- Used Products
- Used Tools
- Testing strategy

Organizational aspects

- Use of human resources
- Prioritization of use cases
- Cost

These decisions need to be documented and can be used to revise decisions if needed.

[5] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 16-17

Architectural design decisions

Architectural views[6]

During the design process, different viewpoints should be taken into consideration.

The 4+1 model defines the following views:

Logical View

- Key abstractions as objects or classes
- Related to system requirements

Process View

- Interacting processes at runtime
- Used for judging performance and availability

Development View

- Components that are to be implemented
- Used by software managers and programmers

Physical View

- Distribution of software across processors
- Used to plan deployment

These views should be used during the design process and for documenting the final architecture.

[6] Sommerville: *Software Engineering* (2016) [3] p. 174

Architectural design decisions

Principles [7]

Divide and conquer:
Complex system is deconstructed top-down to smaller, independent parts

Design to test:
The system is designed in a way that is easy to test

KISS: Keep it simple, stupid
System should be as easy as possible

Yagni: You aren't gonna need it
Over-engineering and speculative requirements that are not directly linked to user demands should be avoided.

DRY: Don't repeat yourself
No code duplicates, otherwise changes will be missed in the duplicates

[7] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 32-33

Architectural design decisions

Principles[8]

Single level of abstract principle:
All statements in a method should have the same level of abstraction

Loose coupling and strong cohesion:
Software components should be loosely coupled among each other and strongly coherent among themselves.

Abstraction:
Modules offer interaction interfaces to reduce negligible knowledge

Information hiding:
Internal data and code of a module are hidden, access is granted through minimal interface.

Separation of concerns:
A good principle for most systems – different objectives of the system should be separated.

[8] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 32-33

Architectural design decisions

Principles[9]

Dependency Inversion Principle:
A higher level module defines the interfaces to be independent of lower levels

Interface Segregation Principle:
clients only receive interfaces they need to use.

Single Responsibility Principle:
A module should only change for a single reason to reduce error risk.

Design by contract:
Relationships between programs is clearly defined in a contract.

Liskovsch's Substitution Principle:
A reference to a parent class should at all times be replaceable with a reference to a child class.

[9] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 32-33

Architectural design decisions

Principles[10]

Principle of Least Astonishment:
Interfaces should be designed to not astonish the user.

Open-Closed Principle:
A module should be open for extension and closed for changes.

Develop against Interfaces, not Implementations:
To build reusable code, use abstract classes or interfaces instead of concrete classes as data types.

Inversion of control:
Stimulus and reaction to that stimulus are separated. The program only defines reaction to stimuli, not the order of commands.

[10] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 32-33

1. Architectural design decisions

2. Architectural patterns

2.1 Structuring patterns
2.2 Adaptable systems
2.3 Distributed systems
2.4 Interactive systems

Architectural patterns

Architectural Patterns[11]

Architectural patterns can be divided in

- **Structuring patterns**
  - Layers
  - Pipes and Filters
  - Repository
- **Adaptable systems**: Plug-In
- **Distributed systems**
  - Client-Service
  - Broker
  - Service-oriented architecture
- **Interactive systems**: Model-View-Controller

[11] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 310-312

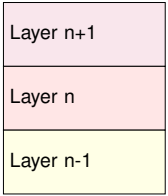
Structuring patterns

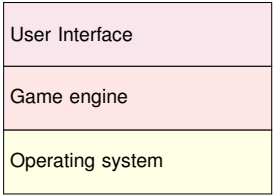
Structuring Patterns

Layers[12,13]

The system is divided into horizontal layers:

- Each layer is associated with a functionality
- A layer can access services provided by the layer below through Interfaces
- The lowest layer provides core functionality
- Every layer is only dependent on the layer directly below





Source: Goll, Koller, and Watzko 2023 [2] p. 317 (adapted)

[12] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 310-312

[13] Sommerville: *Software Engineering* (2016) [3] p. 177- 179

Structuring Patterns

Layers[14,15]

Pros

- Layers directly represent functionality, which is more understandable than the whole system
- Layers can be reused
- Layers are stable and can be standardized
- Dependencies can be minimized
- Layers can be developed in parallel
- Layers can be tested and integrated bottom-up
- Layers can be replaced completely as long as the interface is the same

Cons

- Cleanly separating the right layers can be difficult
- Strict Layering can be too limiting $\rightarrow$ Layer bridging
- Access through layers more time consuming than direct access
- Changes can ripple through multiple layers

[14] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 320-321

[15] Sommerville: *Software Engineering* (2016) [3] p. 177- 179

Structuring Patterns

Layers[16,17]

Layers are often used in

- Projects with Hardware, OS, and application-software
- Embedded systems
- When development is based on an existing system
- When development is spread across several teams for each functionality
- When multilevel security is required

[16] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 321-323

[17] Sommerville: *Software Engineering* (2016) [3] p. 177- 179

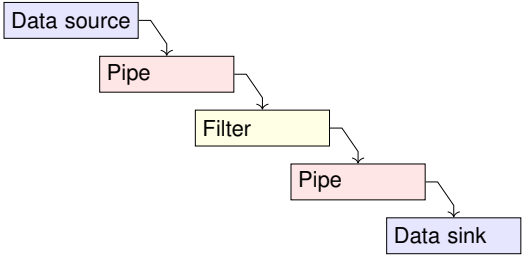
Structuring Patterns

Pipe and Filter[18]

The system is modeled as a chain of sequential data-processes (filters).

Only neighboring filters exchange data and data flows in one direction.

As the data flows, it is transformed by each filter. These transformations can be sequential or parallel.



Source: Goll, Koller, and Watzko 2023 [2] p. 333 (adapted)

[18] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 331-333

Structuring Patterns

Pipe and Filter[19]

Pipes

- Can buffer data
- Asynchronous decoupling
- Serve data later on as needed

Filters

- One Filter for each processing step
- Take data input from previous filter
- Transform the data to output (which serves as input for the next filter)
- Can remove, add, or modify data

All pipes and filters together are a **pipeline**:

- At the beginning of each pipeline, a **data source** provides initial data.
- At the end of every pipeline a **data sink** takes the final output.

Both data source and data sink are connections with the systems environment.

[19] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 332-333

Structuring patterns

Pipe and Filter[20]

Active Filter

- Actively fetches data
- Sends data to next element
- Independent parallel process
- Are coupled with a pipe to even out processing speeds

Passive Filter

- Can be coupled directly
- **Push-principle**:
  - Passively receives data
  - Acts like a void function
- **Pull-principle**:
  - Data is fetched by next element
  - Acts like a function

[20] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 333-335

Structuring patterns

Pipes and Filters[21,22]

Pros

- Flexible
- Reusable
- Enables rapid prototyping
- Decoupled components
- Saving intermediate results is possible but not necessary
- Parallelization to a degree
- Evolution by adding transformations
- Workflow style matches business processes

Cons

- Global data not easily usable
- Passive Filter calls are hard coded
- Bugs are hard to find
- Bottleneck from slowest filter
- Overhead from data-conversion
- Maintenance can ripple through components

[21] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 341-342

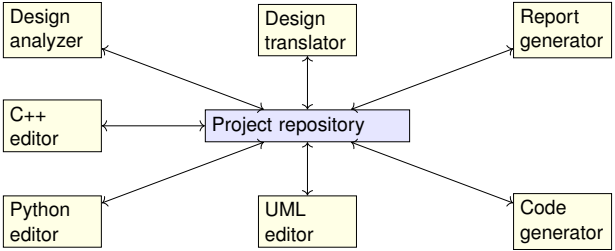
[22] Sommerville: *Software Engineering* (2016) [3] p. 183

Structuring patterns

Repository[23]

A set of interacting components that share the same data can be structured with the Repository pattern. Data is managed centralized in a repository, which all components interact with.

An IDE with different tools could use a repository as following:



Source: Sommerville 2016 [3] p. 180 (adapted)

[23] Sommerville: *Software Engineering* (2016) [3] p. 179-180

Structuring patterns

Repository[24]

Pros

- Independent components
- Data changes from one component available for all others
- Consistent data management

Cons

- Single point of failure
- Bottleneck in communication
- Distribution can be difficult

[24] Sommerville: *Software Engineering* (2016) [3] p. 179-180

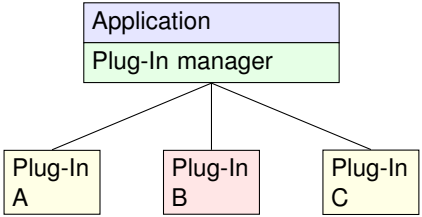
Adaptable systems

Adaptable systems

Plug-In[25]

Plug-Ins are used to create an adaptable system, that can be extended flexibly according to the **Open-Closed principle**.

The system is designed with **extension-points**, where Plug-Ins can be integrated to extend the system during runtime.



Source: Goll, Koller, and Watzko 2023 [2] p. 349 (adapted)

[25] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 348-349

Adaptable systems

Plug-In[26]

Plug-In

- Software-component that provides additional functionality
- Independent of application software
- Complete decoupling of specialized applications
- Usually not executable on its own
- Can be cascaded (layers of plug-ins)

Plug-In manager

- Manages number and type of uses Plug-Ins
- Searches for fitting Plug-Ins
- Instantiates Plug-Ins at runtime as needed

[26] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 349-350

Adaptable systems

Plug-Ins[27]

Pros

- **Separation of concerns**
- Robust
- Extension without knowledge of code
- Lean system
- Easily maintainable
- Easily distributed implementation
- Version management possible
- Independent testing of every Plug-In

Cons

- Initial implementation effort is higher
- Overhead during execution
- Design of common Interface can be hard

Plug-Ins are usually used when multiple user-groups have different requirements that only differ in process details.

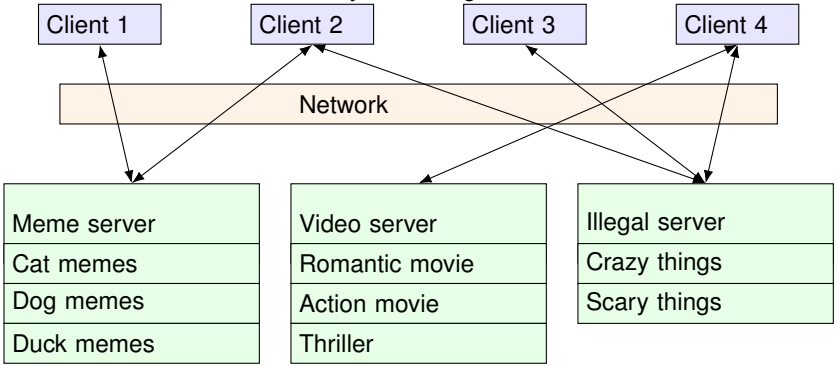
[27] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 358-359

Distributed systems

Distributed systems

Client-server[28]

The system is designed as a set of services, which are delivered by a server.
The clients use these services by accessing the server.



[28] Sommerville: *Software Engineering* (2016) [3] p. 180-182

Distributed systems

Client-server[29,30]

Client

- Calls on server
- **Thin-client**
  - Only one layer on client
  - Usually input or visualization
  - Server processes and stores data
- **Fat-client**
  - Data processing on client
  - Server only used for data storage

Server

- Offers services
- Acts on the clients request

[29] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 325-326, 358-359

[30] Sommerville: *Software Engineering* (2016) [3] p. 503

Distributed systems

Client-server[31,32]

Pros

- Distributed system
- One server serves multiple clients
- Thin client:
  - Client management is simple
  - Minimal requirements for client

Cons

- Single point of failure from each service
- Performance depends on network
- Management can be problematic
- Thin client:
  - Processing load on server
  - Might require more network and server capacity

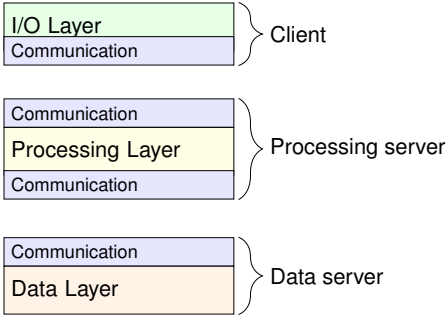
[31] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 325-327, 358-359

[32] Sommerville: *Software Engineering* (2016) [3] p. 503

Distributed systems

Client-server[33,34]

In order to get a more scalable system, a **three-tier-client-server model** may be used.



Source: Goll, Koller, and Watzko 2023 [2] p. 327 (adapted)

[33] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 327

[34] Sommerville: *Software Engineering* (2016) [3] p. 505-506

Distributed systems

Client-server[35]

Additional servers can be added to form a **multi-tier-client-server model**.

Each architecture has different use-cases:

Two-tier with thin clients

- For accessing legacy systems where separating data and application is impractical
- For computationally intensive applications (e.g. compiler)
- For data-intensive applications with low-intensity application processing

[35] Sommerville: *Software Engineering* (2016) [3] p. 506

Distributed systems

Client-server[36]

Two-tier with fat clients

- When application processing is off-the-shelf on client
- When intensive data processing is required (e.g. visualization)
- For mobile applications that require offline functionality

Multi-tier

- For large-scale applications
- When data and application are volatile
- For input data from multiple sources

[36] Sommerville: *Software Engineering* (2016) [3] p. 506

Distributed systems

Broker[37]

In order to separate server and client, the **broker** architecture is used:

- All communication is managed by the broker
- Servers register theirs services
- Clients ask for services
- The broker forwards requests and responses to proxies
- The broker manages all available services

In order to achieve **separation of concerns**, proxies are used between client and broker as well as broker and server:

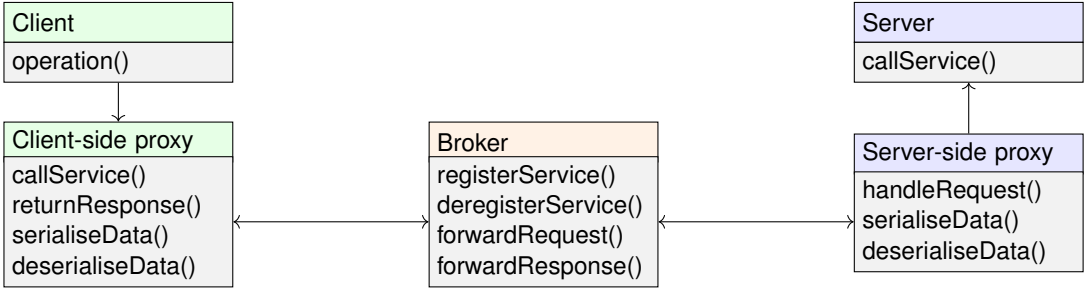
- The client-side proxy serializes data (**marshaling**)
- The server-side proxy deserializes data

With this structure, server and client can be implemented in different programming languages.

[37] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 365-371

Distributed systems

Broker



Source: Goll, Koller, and Watzko 2023 [2] p. 368 (adapted)

Distributed systems

Broker[38]

Pros

- **Separation of concerns**
- Stability (as long as interfaces remain unchanged)
- Spatial independence
- Platform independence

Cons

- Fault tolerance is necessary (e.g. if broker is unavailable)
- Indirect communication leads to performance loss
- Potential bottleneck at broker
- Proxies are dependent on broker

[38] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 385-386

Distributed systems

Broker[39]

The Broker pattern is used when

- Client and Server need to be decoupled
- Components need to access services through service names (instead of location)
- Components change at runtime
- Implementation detail of client and services need to be hidden

[39] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 385-386

Distributed systems

Service-Oriented Architecture[40]

The Service-Oriented Architecture (**SOA**)

- is similar to the broker pattern
- but it focuses on representing the business-view use-cases directly within the architectural design:
  - Different use-cases are isolated as much as possible
  - For every use-case a service is implemented
  - Changes in the use-case lead only to change in one service

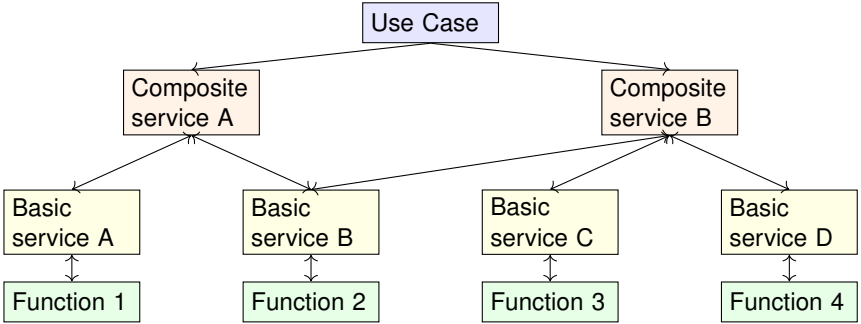
[40] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 389-392

Distributed systems

Service-Oriented Architecture

Specific services are often realized as a combination of other, less abstract **basic services**.

A service that combines other services is called **composite service**.



Source: Sommerville 2016 [3] p. 393 (adapted)

Distributed systems

Service-Oriented Architecture[41]

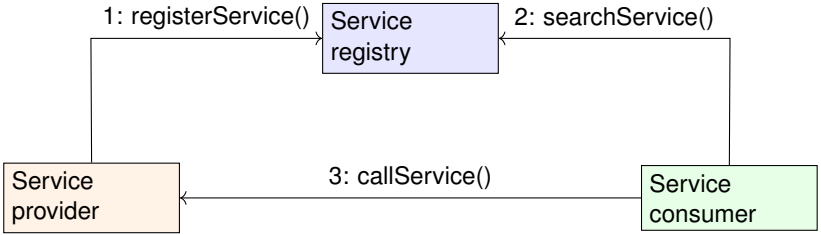
The services are designed to be

- Distributed
- Self-contained
- Stateless
- Loosely coupled
- Exchangeable
- Location transparent
- Platform independent
- Accessible through an interface
- Registered in e.g. a service directory

[41] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 389-391

Distributed systems

Service-Oriented Architecture



Source: Goll, Koller, and Watzko 2023 [2] p. 393 (adapted)

Distributed systems

Service-Oriented Architecture[42]

Pros

- Use cases are mapped to components
- Overview of required services and interfaces
- Complexity is reduced for distributed systems
- Reusability
- Stability against changing requirements

Cons

- Fine-grained services lead to complex structure
- Overhead for communication
- High demands for network
- Only effective for clearly defined and documented use cases

[42] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 396

Interactive systems

Interactive systems

Model-View-Controller[43,44]

Especially for interactive systems, it can be helpful to separate visualization, interaction, and system data.

The Model-View-Controller (**MVC**) pattern structures the system into

Model

- Stores system data
- Manages interactions with system data

View

- Defines how data is visualized
- Manages system output

Controller

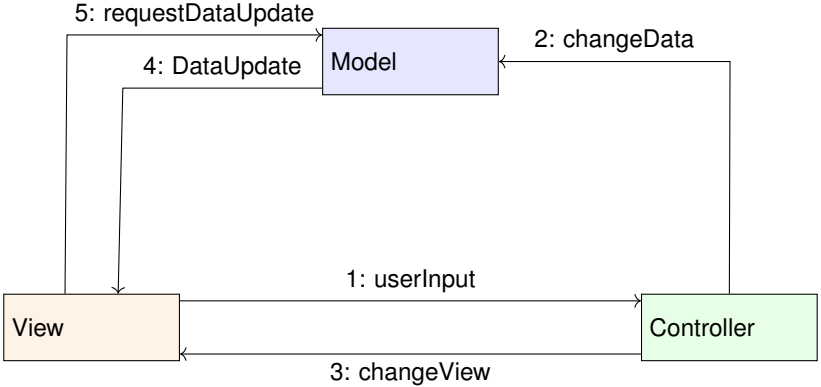
- Manages input
- Handles user interaction

[43] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 422

[44] Sommerville: *Software Engineering* (2016) [3] p. 176

Interactive systems

Model-View-Controller



Source: Sommerville 2016 [3] p. 424 (adapted)

Interactive systems

Model-View-Controller[45,46]

Pros

- Data changes independent of representation
- Supports multiple views on same data
- Changes in one representation shown in all views
- Model can be developed and tested independently of UI
- UI is easily changeable

Cons

- Relatively large overhead for simple interactions
- Dependencies between view and controller
- Increased complexity

[45] Goll, Koller, and Watzko: *Architektur- und Entwurfsmuster der Softwaretechnik* (2023) [2] p. 441-442

[46] Sommerville: *Software Engineering* (2016) [3] p. 176

Questions & References

Comprehensive questions

- Name three principles for architectural design.
- Which views should be taken into consideration for the architecture of a system?
- Explain the role of the Filter in the Pipe and Filter architecture.
- What are the benefits of a Plug-In architecture?
- How can a distributed system be decoupled?

Further reading

- Joachim Goll, Micha Koller, Michael Watzko. Architektur- und Entwurfsmuster der Softwaretechnik. Mit lauffähigen Beispielen in Java. [2], 2023
- Stefan Toth. Vorgehensmuster für Softwarearchitektur. [4], 2019

References I

[1] "IEEE Recommended Practice for Architectural Description for Software-Intensive Systems". In: *IEEE Std 1471-2000* (2000), pp. 1–30.

[2] Joachim Goll, Micha Koller, and Michael Watzko. *Architektur- und Entwurfsmuster der Softwaretechnik. Mit lauffähigen Beispielen in Java*. Springer Vieweg Wiesbaden, 2023.

[3] Ian Sommerville. *Software Engineering*. Always learning. Pearson, 2016.

[4] Stefan Toth. *Vorgehensmuster für Softwarearchitektur*. Carl Hanser Verlag GmbH & Company KG, 2019.